

คู่มือระบบ Thaisausage ERP Integration

- Document version: 1.1.0
- Date: 16 September 2026
- Audience: ทีม IT / Operations, ทีม ERP และนักพัฒนาที่ดูแลระบบ
- Code baseline: working tree ณ วันที่ 16 กันยายน 2026 (commit ล่าสุด 7ce9101 บวกงาน DO callback ที่ยังไม่ commit)
- Production domain: `https://thaisausage.krs.co.th`

คู่มือนี้รวมข้อมูลจากการตรวจสอบโค้ดจริงทั้งหมดไว้ในเล่มเดียว ตั้งแต่การติดตั้ง การตั้งค่า API การ Deploy การดูแลประจำวัน ไปจนถึงผลการตรวจสอบ โปรเจคและข้อจำกัดที่ต้องแก้ก่อนใช้งานจริง หากข้อความในเอกสารอื่นขัดกับคู่มือนี้ ให้ยึดตามโค้ดและคู่มือนี้ (ดูหัวข้อ 14.3)

1. ระบบนี้คืออะไร

Thaisausage เป็น middleware ที่ต้นกลางระหว่าง ERP กับ eVRP ERP แจ้งว่ามี Sales Order (SO) พร้อมส่ง ระบบจะอ่าน SO ฉบับเต็ม ตรวจสอบถูกต้อง กั้นการส่งซ้ำด้วย SQLite แล้วส่งต่อไปยัง eVRP (`POST /v1/orders/import`) นอกจากนี้ระบบยังมีจุดรับ DO เข้าพื้นที่ staging เพื่อตรวจสอบ โดยจะไม่เขียนข้อมูลกลับ ERP

```
ERP SQL Server <-- SELECT-only approved_orders_query (scheduled, IsApprSo = 1) --> map --> validate --+
ERP JSON -----> /erp/orders -----> validate --+
ERP REST <-GET-- /erp/pull -----> map --> validate -----+
                                                    v
                                                    submission guard (SQLite submissions + order_claims) --> eVRP

eVRP DO -----> /vrp/do-received --> do_receipts (staged, never written back to ERP)
```

หลักการสำคัญ:

- ERP เป็นแหล่งข้อมูลหลักของ SO; ระบบใช้ Schedule และ query ที่กรองเฉพาะ `IsApprSo = 1`
- ระบบอ่าน SQL Server แบบ SELECT-only และไม่รับ SQL จาก HTTP request
- เมื่อ `dry_run=true` ระบบจะตรวจและคืน payload แต่ไม่ส่ง eVRP และไม่บันทึกการจอง request/order
- ระบบไม่ resend อัตโนมัติ ถ้าผลลัพธ์ไม่แน่นอน รายการจะเป็น `needs_review` และต้องให้คนตรวจ

2. สถานะปัจจุบัน (16 ก.ย. 2026)

ความสามารถ	มีในโค้ด	พร้อมใช้งานจริง
รับ webhook SO (<code>/erp/hooks/order-ready</code>)	มี	legacy — บันทึก event ได้ แต่ไม่ใช่เส้นทางหลักและไม่ปลุก worker
ส่ง SO แบบ JSON ตรง (<code>/erp/orders</code>)	มี	dry-run เท่านั้น จนกว่าจะผ่าน eVRP UAT
ดึง SO จาก ERP REST (<code>/erp/pull</code>)	มี	ต้องตั้ง <code>erp.base_url</code> และ <code>ERP_TOKEN</code> ก่อน
SQL Server connector	มีโค้ด	ยังรอ query/mapping ที่อนุมัติจาก ERP
Sweep worker แบบ scheduled approved-SO	มี	ปิดอยู่ (<code>sync.enabled=false</code>)
รับ DO จาก eVRP (<code>/vrp/do-received</code>)	มี	ใช่ — staging เท่านั้น
DO staging เดิม (<code>/erp/do-received</code>)	มี	ใช่ — เก็บไว้เพื่อ compatibility
DO writer เข้า <code>tbl_D0hdr / tbl_Dodtl</code>	มีและต่อเข้ากับ callback แล้ว	ไม่ — ปิดอยู่ (<code>do_write.enabled=false</code>) และยังไม่มี mapping จาก DBA
ส่งข้อมูลจริงไป eVRP	มี	**ใช้งานจริงแล้ว** — 18 ก.ย. 2026 ส่งสำเร็จ 10 ใบ (<code>running_code 260916000001-000010</code>)
Swagger UI / OpenAPI	มี	ใช่
COD callback, เขียนกลับ ERP, retry worker, outbox, API ปลด/แก้ SO	ไม่มี	—

ความคืบหน้าตามแผน `process/features/erp-sqlserver/` : Phase 01 (Discovery) อยู่ในสถานะ TESTING เพราะยังขาด ODBC driver และ SO ตัวอย่างที่ได้รับอนุมัติ ส่วน Phase 02–06 ยังไม่เริ่ม

Production ที่ติดตั้งอยู่ตาม Operations Guide คือ VPS `krs-cloud` ใช้ Docker Compose + Caddy และตั้ง `dry_run=true` , `sync.enabled=false`

3. โครงสร้างโปรเจค

Path	หน้าที่
thaisausage/__main__.py	จุดเริ่มโปรแกรม: โหลด config, สร้าง service, HTTP server และ thread sql-sweep
thaisausage/api.py	HTTP handler: auth, จำกัด body, routing, แปลง error เป็น HTTP status
thaisausage/service.py	business logic: submission guard, webhook inbox, DO staging, sweep และ SQLite schema
thaisausage/contracts.py	กติกา validate ของ standard SO JSON
thaisausage/connectors.py	HTTPS client ไป ERP REST และ eVRP รวมถึงฟังก์ชัน field mapping (mapped , lookup)
thaisausage/sqlserver.py	ODBC connection string, SELECT guard, fetch_orders และจัดกลุ่มแถวเป็น SO
thaisausage/do_writer.py	DO writer เข้า ERP: สร้าง INSERT แบบ parameterized, transaction เดียว, ปิดเป็นคำเริ่มต้น
thaisausage/discover.py	อ่าน metadata ของ ERP แบบ SELECT-only (ชนิดข้อมูล, key, trigger, FK) และค้นหาว่าคอลัมน์นี้อยู่ตารางไหน
thaisausage/preflight.py	ตรวจ config + ข้อมูล SO ของเราก่อนส่ง โดยไม่ติดต่อ eVRP
thaisausage/config.py	อ่าน .env และ config JSON พร้อมตรวจค่าบังคับ
config/example.json	แม่แบบ config (สำเนาจริงคือ config/local.json ซึ่งถูก ignore)
config/do-write-template.json	แม่แบบ mapping ของ DO writer ครบ 55 + 24 คอลัมน์ตามลำดับของ ERP (ยังเว้น path ให้เติม)
examples/erp-order.json	ตัวอย่าง request สำหรับ /erp/orders
docs/openapi.json	OpenAPI 3.0 ที่ใช้แสดงผลใน /docs
deploy/	Dockerfile, docker-compose.yml, Caddyfile และ vps.env.example
tests/	unittest: integration (HTTP + mock upstream) และ SQL config
process/	แผนงาน, รายงาน phase และ context ของ workflow

ระบบใช้ Python 3.9 ขึ้นไปและ standard library เป็นหลัก ต้องใช้ pyodbc ร่วมกับ Microsoft ODBC Driver 18 เฉพาะเมื่อเชื่อม SQL Server (Docker image ติดตั้งให้แล้ว)

4. เริ่มใช้งานบนเครื่อง local

รันจาก root ของโปรเจค เพราะไฟล์ .env ถูกอ่านจาก working directory ปัจจุบัน:

```
cp config/example.json config/local.json
# สร้างไฟล์ .env อย่างน้อยต้องมี THAISAUUSAGE_API_KEY
python3 -m thaisausage --config config/local.json
```

เมื่อเริ่มสำเร็จจะเห็นข้อความ Thaisausage API http://0.0.0.0:8080 dry_run=True

```
curl http://127.0.0.1:8080/health
curl -X POST http://127.0.0.1:8080/api/v1/erp/orders \
  -H "Authorization: Bearer $THAISAUUSAGE_API_KEY" \
  -H 'Content-Type: application/json' \
  --data-binary @examples/erp-order.json
```

ระบบอ่าน config เฉพาะตอนเริ่ม ถ้าแก้ config/local.json หรือ .env ต้อง restart ทุกครั้ง

5. ตัวแปร environment (secrets)

Variable	ต้องใช้เมื่อ	หน้าที่
THAISAUUSAGE_API_KEY	ทุกกรณี (ถ้าไม่มี ระบบจะไม่เริ่ม)	Bearer key ที่ ERP ใช้เรียก API
VRP_TOKEN	dry_run=false (ถ้าไม่มี ระบบจะไม่เริ่ม)	ส่งไป eVRP ใน header X-Token
ERP_TOKEN	ใช้ /erp/pull	token สำหรับเรียก ERP REST
ERP_SQLSERVER_CONNECTION_STRING	sweep ทำงาน	ส่วน server/database/encryption ของ ODBC string
ERP_SQL_USER / ERP_SQL_PASSWORD	sqlserver.auth_mode=sql	credential แบบอ่านอย่างเดียว ระบบต่อท้ายให้ตอน connect

กติกาของ .env :

- ระบบรองรับรูปแบบ KEY=value บรรทัดที่ขึ้นต้นด้วย # เป็น comment และตัดเครื่องหมายคำพูดครอบคำออกให้
- ถ้า environment ของ shell หรือ container มีค่าอยู่แล้ว ค่านั้นมีสิทธิ์เหนือค่าในไฟล์
- .env และ deploy/vps.env อยู่ใน .gitignore ห้าม commit ห้ามใส่ในเอกสาร และห้ามถ่ายภาพหน้าจอที่มีค่าเหล่านี้

- ห้ามใส่ UID หรือ PWD ใน connection string เพราะระบบจะปฏิเสธ ให้ใช้ตัวแปรแทน
- บน VPS ให้เก็บที่ /opt/thaisausage/.env และตั้ง permission เป็น 600
- deploy/vps.env ใช้เก็บข้อมูล SSH ของผู้ดูแล (VPS_HOST , VPS_PORT , VPS_USER , VPS_SSH_KEY) และไม่ได้ถูกโหลดเข้า container

6. การตั้งค่า `config/local.json`

ระบบใช้ไฟล์สองชั้นแยกกันเสมอ: .env เก็บ secret อย่างเดียว ส่วน config/local.json เก็บพฤติกรรมของระบบและห้ามมี secret เด็ดขาด (ใส่ได้แค่ *ชื่อ* ตัวแปร environment) ทั้งสองไฟล์อยู่ใน .gitignore และ deploy/.dockerignore ส่วน Docker image คัดลอกเฉพาะ config/example.json ขณะที่ config/local.json ถูก mount แบบ read-only ตอนรัน

บน VPS ต้องมีอย่างน้อย: .env → THAISAUUSAGE_API_KEY, VRP_TOKEN, ERP_SQLSERVER_CONNECTION_STRING, ERP_SQL_USER, ERP_SQL_PASSWORD และ config/local.json → dry_run, sync.enabled, sync.interval_seconds, sqlserver.approved_orders_query, sqlserver.field_map, sqlserver.item_field_map, do_write.enabled (ต้องเป็น false) พร้อม mapping ของ do_write

Key	ค่าเริ่มต้นใน example	ความหมาย
host, port	0.0.0.0, 8080	address ที่ HTTP server รอรับ
database	data/integration.sqlite3	ไฟล์ SQLite (path สัมพันธ์กับ working directory)
api_key_env	THAISAUUSAGE_API_KEY	ชื่อตัวแปรที่เก็บ API key
dry_run	true	ต้องเป็น boolean เท่านั้น false คือส่งจริง
sync.enabled	false	เปิดหรือปิด scheduled worker ที่ตั้ง SO ที่อนุมัติแล้ว
sync.interval_seconds	60	รอบการทำงานของ scheduled worker
vrp.base_url	https://vrp.oneplatformth.com/api	ต้องเป็น HTTPS
vrp.token_env, vrp.timeout_seconds	VRP_TOKEN, 30	token และ timeout ของ eVRP
erp.base_url, orders_path	ว่าง, /api/orders	ERP REST สำหรับ /erp/pull
erp.token_env, auth_header, auth_prefix	ERP_TOKEN, Authorization, Bearer	วิธีส่ง token ไป ERP
erp.orders_list_path, items_path	data.orders, items	ตำแหน่ง array ใน JSON ของ ERP
erp.field_map, item_field_map	map ชื่อเดียวกัน	key = field ปลายทาง, value = path ใน ERP
sqlserver.connection_string_env, username_env, password_env	ชื่อตามหัวข้อ 5	ชื่อตัวแปร secret
sqlserver.auth_mode	sql	sql หรือ integrated (Trusted_Connection)
sqlserver.connect_timeout_seconds, query_timeout_seconds	10, 30	timeout ของ ODBC
sqlserver.approved_orders_query	ว่าง	SELECT ที่ผ่านการ review แล้ว ต้องมีคำว่า IsApprSo (เช่น WHERE IsApprSo = 1) มิฉะนั้นระบบปฏิเสธก่อนเชื่อมต่อ
sqlserver.totals_query	ไม่มี	ไม่บังคับ — SELECT คืน order_no, total_amount, total_actual_amount ให้ preflight ใช้กระทบยอดกับหัว SO
sqlserver.order_key	order_no	ชื่อคอลัมน์ที่ใช้จัดกลุ่มแถวเป็น SO
sqlserver.field_map, item_field_map	ว่าง	map คอลัมน์ SQL ไปเป็น standard JSON
do_write.enabled	false	เปิดการเขียน DO เข้า ERP ห้ามเปิดจนกว่าจะผ่าน UAT และ sign-off
do_write.header_table, detail_table	ว่าง	ชื่อตารางปลายทางที่ผ่านการ review
do_write.connection_string_env, username_env, password_env	ERP_SQLSERVER_WRITE_*, ERP_SQL_WRITE_*	credential สำหรับเขียน ต้องคนละชุดกับบัญชี read-only
do_write.header_columns, detail_columns	ว่าง	mapping รายคอลัมน์ — คัดลอกโครงจาก config/do-write-template.json แล้วเติม path (ดู docs/do-insert-contract.md)
do_write.detail_line_start	1	เลขเริ่มต้นของ Slno

Key ต่อไปนี้อยู่ใน example แต่ได้ตั้ยังไม่ได้อ่าน จึงยังไม่ีผลกับการทำงานจริง: source.*, sqlserver.driver, sqlserver.encrypt, sqlserver.trust_server_certificate, sqlserver.approved_objects, sqlserver.approved_query_set ค่า driver และ encryption ที่ใช้จริงจะมาจาก ERP_SQLSERVER_CONNECTION_STRING เท่านั้น

Mapping รองรับ path ซ้อนกันด้วยจุด เช่น "customer.code": "buyer.customer_code" ถ้าไม่พบ path ระบบจะใส่ค่า null ระบบจะไม่แปลงชนิดข้อมูลภายใน ส่วนลด หรือหน่วยให้

7. API reference

7.1 การยืนยันตัวตนและข้อจำกัดทั่วไป

Endpoint สาธารณะที่ไม่ต้องใช้ key ได้แก่ GET/HEAD /health, GET/HEAD /openapi.json, GET/HEAD /docs และ GET /favicon.ico (ตอน 204) ส่วน endpoint อื่นทั้งหมดต้องส่ง header ดังนี้:

```
Authorization: Bearer <THAISAUUSAGE_API_KEY>
Content-Type: application/json
```

ข้อจำกัดของ POST ทุกตัว:

- Content-Type ต้องเป็น application/json และต้องมี Content-Length ขนาด 1 ถึง 1,048,576 bytes
- ไม่รองรับ Transfer-Encoding (chunked) หากส่งมาจะได้ 422
- body ต้องเป็น JSON object และห้ามมี NaN หรือ Infinity
- connection มี timeout 35 วินาที
- ระบบไม่เขียน access log เพื่อไม่ให้ข้อมูลลูกค้าหรือ header รั่วไหล

7.2 สรุป endpoint

Method / Path	Auth	สำเร็จ	หน้าที่
GET /health	ไม่ต้อง	200	{status, dry_run} ตรวจสอบว่า process ทำงานอยู่ ไม่ได้ตรวจ SQL หรือ eVRP
GET /docs	ไม่ต้อง	200	Swagger UI (โหลดไฟล์จาก unpkg.com)
GET /openapi.json	ไม่ต้อง	200	OpenAPI contract
POST /api/v1/erp/hooks/order-ready	ต้อง	202	legacy — บันทึก event ลง erp_hooks เท่านั้น ไม่ปลุก scheduled worker และไม่ใช้เส้นทางหลัก
POST /api/v1/erp/orders	ต้อง	200/202	รับ standard SO JSON แล้ว validate และส่งต่อ
POST /api/v1/erp/pull	ต้อง	200/202	ดึง ERP REST หนึ่งครั้ง แล้ว map, validate และส่งต่อ
POST /api/v1/vrp/do-received	ต้อง	202	eVRP ส่ง DO เข้า staging และเขียน ERP ถ้าเปิด do_write.enabled
POST /api/v1/vrp/do-updated	ต้อง	202	eVRP ส่ง DO ฉบับแก้ไข ปรับเอกสารเดิมใน ERP ถ้าเปิด do_write.update_enabled
GET /api/v1/submissions/{request_id}	ต้อง	200/404	ผลที่บันทึกไว้ของการส่งจริง

7.3 Webhook: `POST /api/v1/erp/hooks/order-ready`

```
{
  "event_id": "ERP-SO-20260912-000001",
  "event_type": "sales_order.ready",
  "source_id": "main-erp",
  "company_id": "THAI",
  "order_no": "SO-0001",
  "changed_at": "2026-09-12T10:00:00+07:00"
}
```

Field	บังคับ	กติกา
event_id	ใช่	ไม่ว่างและยาวไม่เกิน 120 ตัว ควรใช้เฉพาะ A-Z a-z 0-9 . _ : - (ดู F5)
event_type	ใช่	sales_order.ready หรือ sales_order.changed
source_id	ใช่	รหัสแหล่ง ERP
order_no	ไม่บังคับใน API แต่ควรส่งทุกครั้ง	ใช้เป็นพารามิเตอร์ของ SQL query (ดู F4)
company_id, changed_at	ไม่	บันทึกไว้เพื่อใช้ตรวจสอบ ปัจจุบัน sweep ยังไม่ได้ใช้

ครั้งแรกระบบจะตอบ 202 {"event_id", "state": "received", "sweep": "queued"} ถ้าส่ง event_id เดิมพร้อม body เดิมซ้ำจะได้ 202 {"event_id", "state": "สถานะปัจจุบัน", "replayed": true} หาก body ต่างกันจะตอบ 409 และไม่ดำเนินการซ้ำ ทุกการแก้ไข SO จึงต้องใช้ event_id ใหม่

การตอบ 202 หมายความว่าบันทึก trigger แล้วเท่านั้น ไม่ได้แปลว่า SO ถึง eVRP แล้ว

7.4 ส่ง SO ตรง: `POST /api/v1/erp/orders`

ดูตัวอย่างเต็มได้ใน examples/erp-order.json กติกา validate มีดังนี้ (thaisausage/contracts.py):

Field	กติกา
request_id	1-120 ตัว ใช้ได้เฉพาะ A-Z a-z 0-9 . _ : -
orders	array จำนวน 1-100 รายการ
order_no	บังคับ ยาวไม่เกิน 100 และห้ามซ้ำใน batch
order_date	YYYY-MM-DD แบบ string
delivery_date	null, "" หรือ YYYY-MM-DD
payment_in_day	null หรือตัวเลขที่ ≥ 0 โดย ** 0 หมายถึง COD** และ null หมายถึงไม่ใช่ COD; ห้ามแปลงค่าว่างเป็น 0 เด็ดขาด (ดู 12.3)
pickup_hub_code	บังคับ
delivery_point_code / shipping_address	ต้องมียาวน้อยหนึ่งค่า
customer.code	บังคับ ยาวไม่เกิน 50
items	array จำนวน 1-2000 รายการ
items[].item_code	บังคับ ยาวไม่เกิน 50
items[].description	บังคับ — eVRP กำหนดให้ทุกบรรทัดต้องมีคำอธิบาย ระบบตรวจให้ก่อนส่ง
items[].quantity	JSON number ที่ > 0
items[].unit_price	JSON number ที่ ≥ 0 (สินค้าแถมใช้ราคา 0 และแยกบรรทัด)
items[].cbm, nw	null หรือตัวเลขที่ ≥ 0

Field ที่ระบบยังไม่ validate ได้แก่ customer.name, description, pickup_hub_name และหน่วยสินค้า eVRP จะตรวจ master data อีกครั้งเมื่อส่งจริง ตัวเลขที่ส่งมาเป็น string เช่น "100" จะถูกปฏิเสธ

ผลลัพธ์:

- dry-run ตอบ 200 {state: "validated", dry_run: true, order_count, mapped_payload} โดยไม่บันทึกอะไรลงฐานข้อมูล
- live ตอบ 200 {state: "sent", vrp: {...}} เมื่อ eVRP ยืนยัน success: true หรือ 202 {state: "needs_review", reason} เมื่อผลไม่แน่นอน
- ส่ง request_id เดิมพร้อม payload เดิมจะได้ผลเดิมกลับมาพร้อม replayed: true
- request_id เดิมที่มี payload ต่างกัน หรือ order_no ที่เคยถูกจองภายใต้ request_id อื่น จะได้ 409 และระบบจะไม่ส่งทั้ง batch

7.5 ดึงจาก ERP REST: `POST /api/v1/erp/pull`

```
{"request_id": "ERP-PULL-0001", "query": {"page": 1}}
```

query ต้องเป็น object ที่มีแต่ค่า scalar ระบบจะแปลงเป็น query string แล้ว GET ไป erp.orders_path หนึ่งครั้ง ไม่มีการใส่หน้าอัตราโน้มนำ ถ้าไม่มีรายการจะตอบ 200 {state: "empty"} ถ้ามีรายการระบบจะทำงานเหมือน /erp/orders ถ้าเรียก ERP ไม่สำเร็จจะได้ 502 ข้อควรระวังคือแม้อยู่ใน dry-run ระบบก็ยังเรียก ERP จริง

7.6 DO callback: `POST /api/v1/vrp/do-received`

eVRP เรียก endpoint นี้เมื่อสร้าง DO เสร็จแล้ว ระบบจะตรวจสอบ receipt_id / do_no, บันทึก payload ลง do_receipts และตอบ 202. การรับซ้ำด้วยข้อมูลเดิมเป็น replay ที่ปลอดภัย ส่วนข้อมูลเดิมแต่ payload เปลี่ยนจะตอบ 409.

DO ที่รับเข้ามาจะถูกเก็บลง staging เสมอ ก่อน แล้วจึงเขียนเข้า ERP ถ้าเปิด do_write.enabled — ถ้าปิดอยู่ ต้องสั่งเขียนทีละใบด้วย python -m thaissausage.write_do (ดูหัวข้อ 9.1) การเขียนใช้ credential คนละชุดกับบัญชีอ่าน SO เสมอ

ต้องมี receipt_id หรือ do_no (ยาวไม่เกิน 120) ระบบเก็บ payload เดิมพร้อม hash แล้วตอบ 202 รูปแบบของ header / details อยู่ใน docs/eVRP_DO_Callback_Spec.md §9

ตัวอย่าง request:

```
{"receipt_id": "VRP-D0-0001", "do_no": "D0-0001", "status": "completed",
"header": {"do_no": "D0-0001", "dodate": "2026-09-22", "cust_code": "CT-MS-BKK-2092"},
"details": [{"item_code": "A20-002", "qty": 10, "units": "ลัง", "amount": 5950.00}]}
```

ตัวอย่าง response ครั้งแรกและตอนส่งซ้ำ:

```
{"receipt_id": "VRP-D0-0001", "receipt_key": "eVRP:VRP-D0-0001", "source": "eVRP", "do_no": "D0-0001", "transaction_no": 900001,
"state": "staged", "erp_write": true, "do_write": {"state": "inserted", "transaction_no": 900001}}
{"receipt_id": "VRP-D0-0001", "receipt_key": "eVRP:VRP-D0-0001", "source": "eVRP", "do_no": "D0-0001", "transaction_no": 900001,
"state": "staged", "erp_write": false, "replayed": true}
```

transaction_no ที่ระดับบนสุดคือเลขเอกสารใน ERP ซึ่งอ่านจากบันทึกการเขียนของเราเองทุกครั้ง เป็น null จนกว่าจะเขียนสำเร็จ และเป็นคีย์ที่ eVRP ใช้อ้างอิงเวลาคุยกับทีม ERP ส่วน do_no สะท้อนกลับไปเพราะเป็นตัวระบุเอกสารตอนแก้ไขภายหลัง

เมื่อเปิด `do_write.enabled=true` ระบบจะพยายามเขียน `tbl_D0hdr / tbl_Dodtl` ต่อทันทีหลัง staging แล้วรายงานผลในฟิลด์ `do_write` หลักการสำคัญคือ ****staging ที่สำเร็จตอบ 202 เสมอ**** ปัญหาของ writer ไม่เปลี่ยน HTTP status และ `erp_write` เป็น boolean ที่เป็น `true` เฉพาะตอน `do_write.state = inserted` เท่านั้น

<code>do_write.state</code>	ความหมาย	<code>erp_write</code>
inserted	เขียน Header/Detail สำเร็จและ commit แล้ว	true
disabled	<code>do_write.enabled=false</code>	false
skipped	ไม่มีไครออลเลข TransactionNo ได้ — payload ไม่ส่งมา และ writer ไม่ได้ตั้งให้จัดสรรเอง	false
rejected	mapping/ข้อมูลไม่ผ่าน validate ไม่มี SQL ถูกส่ง	false
failed	SQL Server ปฏิเสธค่า (เช่นข้อความยาวเกินคอลัมน์) ก่อน commit และ rollback แล้ว — <code>reason</code> บอกตารางและคอลัมน์	false
conflict	<code>do_no</code> หรือ <code>transaction_no</code> ถูกใช้โดย receipt อื่น	false
needs_review	ผลไม่แน่นอน เช่น timeout ระหว่าง commit	false

`source` เป็น eVRP เมื่อเข้าทาง `/api/v1/vrp/do-received` และเป็น erp เมื่อเข้าทาง endpoint เดิม (เส้นทาง ERP ไม่เรียก writer เลย) identity ที่ใช้จริงคือ `receipt_key = "<source>:<receipt_id>"` ดังนั้น receipt เลขเดียวกันจากสองช่องทางจะไม่ชนกัน การรับข้างของ identity ภายใน transaction เดียว (BEGIN IMMEDIATE) callback ที่มาพร้อมกันจึงได้ 202 พร้อม `replayed: true` ไม่ใช่ 500 eVRP ต้องใช้ `receipt_id` เดิมทุกครั้งที่ส่ง DO ใบเดิมซ้ำเส้นทางฝั่ง ERP (`/api/v1/erp/do-received`) ไม่เรียก writer เลยไม่ว่ากรณีใด

7.7 DO ฉบับแก้ไข: ``POST /api/v1/vrp/do-updated``

ใช้เมื่อ DO ที่เคยส่งมาแล้วมีการเปลี่ยนแปลง แยกเส้นทางจากการสร้างเพราะเส้นสร้างถูกออกแบบให้ ****ปฏิเสธ**** การแก้ (409 เมื่อ `receipt_id` เดิมมาพร้อมข้อมูลใหม่) ซึ่งเป็นเกราะกันข้อมูลเพี้ยนที่ต้องคงไว้

eVRP ส่ง `receipt_id` ใหม่ + `do_no` เดิม + `header / details` ฉบับสมบูรณ์ ระบบจะ

1. เก็บ payload ลง staging เหมือนทุก callback 2. ค้น TransactionNo จากตาราง `do_writes` ด้วย `do_no` — ****ไม่รับเลขจาก payload**** เพื่อไม่ให้ค่าจากภายนอกชี้ไปเอกสารใบอื่นใน ERP 3. ตรวจสอบเอกสารยังแก้ไขได้ (IsApproved / IsClosed / IsAcc ต้องเป็น 0 และต้องมีแถวเดียว) 4. UPDATE หัวเอกสารในที่เดิมแล้ว DELETE + INSERT รายการสินค้าใหม่ทั้งหมด ในทรานแซกชันเดียว

<code>do_update.state</code>	ความหมาย	<code>erp_write</code>
updated	แก้และ commit สำเร็จ	true
disabled	<code>do_write.update_enabled=false</code>	false
rejected	<code>do_no_not_written</code> , เอกสารถูกอนุมัติ/ปิด/ลงบัญชี, หรือ mapping ไม่ผ่าน	false
conflict	<code>receipt_id</code> เดิมมาพร้อมข้อมูลต่าง	false
needs_review	ผลไม่แน่นอน เช่น timeout ระหว่าง commit	false

บันทึกอยู่ในตาราง `do_updates` แยกจาก `do_writes` การส่งซ้ำด้วย `receipt_id` และข้อมูลเดิมจะได้ `replayed: true` โดยไม่แก้ซ้ำ

ข้อตกลงที่มีผล (ยืนยัน 22 ก.ย. 2026)

- ****ข้อมูลจาก eVRP ถือเป็นฉบับจริง**** ถ้าทีม ERP แก้เอกสารด้วยมือไว้ แล้ว eVRP ส่งฉบับแก้มาทีหลัง การแก้ของทีม ERP จะถูกทับ วิธีเลือกคือให้ ERP Approve หรือ Close เอกสารนั้น ระบบจะปฏิเสธการแก้ทันที
- ****การยกเลิก DO เป็นสิทธิ์ของทีม ERP เท่านั้น**** ระบบนี้ไม่ยกเลิกเอกสารให้ไม่ว่ากรณีใด และไม่มี endpoint สำหรับยกเลิก

7.8 ตรวจสอบสถานะ: ``GET /api/v1/submissions/{request_id}``

endpoint นี้คืนผลที่บันทึกไว้ใน SQLite ซึ่งมีเฉพาะการส่งในโหมด live ดังนั้นในโหมด dry-run จะได้ 404 เสมอ

7.9 HTTP status

HTTP	ความหมาย	สิ่งที่ผู้เรียกควรทำ
200	validated / sent / empty / replay ของผลที่เสร็จแล้ว	บันทึกผล
202	บันทึก hook หรือ DO แล้ว หรือผลเป็น sending / needs_review	ห้ามถือว่าส่งถึง eVRP แล้ว
401	key ไม่มีหรือไม่ถูกต้อง	แก้ credential แล้วส่ง identity เดิมซ้ำ
404	path ไม่ถูกต้อง หรือไม่พบ submission	ตรวจ URL หรือ request_id
409	identity เดิมแต่ข้อมูลต่าง	หยุด automation แล้ว reconcile
422	JSON, header หรือ field ไม่ถูกต้อง	แก้ payload ถ้าข้อมูลเปลี่ยนให้ใช้ identity ใหม่
502	เรียก ERP REST ไม่สำเร็จ (/erp/pull)	retry ตามนโยบายของ operator
500	internal error	แจ้งผู้ดูแลระบบ

7.10 สถานะ submission และ hook

State	ใช้กับ	ความหมาย
received	hook	บันทึกแล้ว รอ sweep
validated	submission, hook	ผ่าน validate ใน dry-run โดยยังไม่ได้ส่งจริง
sending	submission, hook	จองแล้ว กำลังส่ง ถ้าค้างอยู่แสดงว่า process หยุดกลางทาง ต้องตรวจเอง
sent	submission, hook	eVRP ตอบ success: true
needs_review	submission, hook	ผลไม่แน่นอน: upstream_response_not_confirmed , upstream_http_NNN , upstream_outcome_unknown , connector_error
review	hook	sweep ไม่พบ SO หรือเกิด error (ระบบไม่ได้เก็บเหตุผลไว้ ดู F6)
staged	DO	เก็บเข้า staging แล้ว

8. การทำงานของ scheduled worker

thread sql-sweep เริ่มพร้อม HTTP server และทำงานเป็นรอบดังนี้:

- รอจนครบ sync.interval_seconds แล้วทำงานหนึ่งรอบ ไม่มีสิ่งใดมาปลุกก่อนกำหนด (hook ไม่ปลุก worker แล้ว)
- ถ้า sync.enabled=false ระบบข้ามรอบนั้นไป แต่ HTTP API ยังทำงานตามปกติ
- เรียก sqlserver.approved_orders_query ที่ผ่านการ review แบบ SELECT-only ไม่มีพารามิเตอร์ และไม่รับ SQL จาก HTTP
- query ต้องกรองเฉพาะ SO ที่อนุมัติแล้ว (IsApprSo = 1) ระบบเชื่อผลลัพธ์ของ query นี้ จึงไม่มีตัวกรองซ้ำในโค้ด
- แต่ละ SO ใช้ request_id = "SCHEDULE-<hash ของ order_no>-<hash ของ เนื้อข้อมูล>" ข้อมูลเดิมได้ id เดิม ข้อมูลที่แก้แล้วได้ id ใหม่ (ดู 12.3)
- ระวัง TOP (n) ใน query จำกัด **จำนวนแถว** ไม่ใช่จำนวนใบ ถ้าตั้งน้อยเกินไป ใบที่เกินโควตาจะไม่ถูกส่งเลยแม้รอบถัดไป (ปริมาณจริงประมาณ 108 ใบ / 293 แถวต่อวัน)
- SO ที่ส่งสำเร็จแล้วได้ skipped เหตุผล already_sent ; SO ที่ครั้งก่อนค้างที่ sending / needs_review จะได้ needs_review เหตุผล prior_attempt_* และระบบไม่ส่งซ้ำให้เอง
- SO ที่มีบรรทัดสินค้าไม่มี item_code จะได้ review เหตุผล detail_item_code_missing และไม่ถูกส่งทั้งใบ
- ถ้าอ่าน query ทั้งหมดไม่ได้ (เช่นแถวไม่มี order key) รอบนั้นคืนผลเดียวเป็น review โดยไม่โยน exception ออกไป
- แต่ละ SO ถูกแยกจากกัน ถ้ารายการใดฟังจะได้ review พร้อมเหตุผลและเขียน log แบบ redact ส่วนรายการที่เหลือยังส่งต่อได้ตามปกติ
- แถวที่ไม่มี order_no จะได้ review เหตุผล order_no_missing โดยไม่พยายามส่ง

พฤติกรรมที่ผู้ดูแลต้องรู้:

- เมื่อ dry_run=true ระบบจะ validate และ preview เท่านั้น ผลลัพธ์เป็น preview เหตุผล dry_run_preview ไม่มีการส่ง eVRP และไม่มีการจอง order_no
- SO ที่ส่งไปแล้วและถูกแก้ไขภายหลังจะชนการจอง order_no กลายเป็น review ต้องแก้ที่ eVRP ด้วยมือ เพราะยังไม่มี contract สำหรับ update หรือ cancel
- erp_hooks และ sweep_hooks ยังอยู่ในโค้ดเพื่อ compatibility แต่ไม่ได้ถูกเรียกจาก worker ปัจจุบัน

8.1 จำกัดจำนวนใบต่อรอบ (`sync.max\_new\_orders\_per\_cycle`)

ตั้งเป็นตัวเลขเพื่อจำกัดว่าในหนึ่งรอบกวาดจะส่งใบใหม่ออกไปได้กี่ใบ ใบที่เคยจัดการแล้วไม่นับ งานค้างจึงทยอยไหลออกหลายรอบแทนที่จะไปถึง eVRP พร้อมกันทีเดียว
จำเป็นตอนขยายขอบเขตของ query เช่นวันที่ 29 ก.ย. 2026 ที่เลิกกรองเฉพาะบริษัท TST — ถ้าไม่มีเพดาน รอบแรกจะส่งใบที่ไม่เคยถูกส่งทั้งหมดในครั้งเดียว (ตอนนั้นคือ 1,331 ใบ) ถ้าตั้งค่าไว้ 50 และรอบกวาด ทุก 60 วินาที งานค้างเท่านั้นจะใช้เวลาประมาณ 27 นาทีทั้งหมด และถ้าปลายทางปฏิเสธ จะเห็นตั้งแต่ 50 ใบแรกแทนที่จะเห็นตอนฟังทั้งพัน
ไม่ใช่ค่า = ไม่จำกัด

9. SQL Server connector

การประกอบ connection string: ระบบนำ ERP_SQLSERVER_CONNECTION_STRING มาต่อท้ายด้วย UID={user};PWD={password} (escape คำด้วยวงเล็บปีกกา) เมื่อใช้ auth_mode=sql หรือต่อท้าย Trusted_Connection=yes เมื่อใช้ integrated

```
DRIVER={ODBC Driver 18 for SQL Server};SERVER=erp-db.example,1433;DATABASE=ERP;Encrypt=yes;TrustServerCertificate=no;Connection Timeout=10
```

ก่อนเปิด connection ทุกครั้ง ระบบบังคับสัญญาของ query ดังนี้ (ตรวจด้วยข้อความ ไม่ใช่ SQL parser จึงเลือกปฏิเสธไว้ก่อนเมื่อไม่มั่นใจ):

- ต้องเป็นคำสั่งเดียว ห้ามมี ;
- ห้ามมี comment -- หรือ /* */ เพื่อไม่ให้ซ่อนเงื่อนไข
- ต้องขึ้นต้นด้วย SELECT และห้ามมีคำว่า INSERT, UPDATE, DELETE, MERGE, EXEC, ALTER, DROP, CREATE, TRUNCATE, GRANT, REVOKE, DENY, BACKUP, RESTORE, WAITFOR, BULK, OPENROWSET, OPENQUERY, OPENDATASOURCE หรือ INTO
- เฉพาะ approved_orders_query ต้องมีเงื่อนไข IsApprSo = 1 ที่มองเห็นได้ และห้ามเทียบ IsApprSo กับค่าอื่น เช่น = 0 , < 1 หรือ >= 1
- ถ้า view กรอง approved ไว้แล้ว ก็ยังต้องเขียน IsApprSo = 1 ใน query เพื่อให้ตรวจสอบได้จากไฟล์ config

guard นี้ตรวจข้อความ ไม่ใช่กลไกความปลอดภัยหลัก บัญชี SQL จึงยังต้องมีสิทธิ์ SELECT เฉพาะ view ที่อนุมัติเท่านั้น (ดู F11)

การแปลงแถว SQL เป็น SO:

- แถวผลลัพธ์จะถูกจัดกลุ่มตามคอลัมน์ order_key โดยใช้แถวแรกของกลุ่ม map header ด้วย field_map
- ทุกแถวถูก map ด้วย item_field_map ; ถ้าบรรทัดใดไม่มี item_code ที่ใช้ได้ ทั้ง SO จะถูกทำเครื่องหมาย rejected_reason = detail_item_code_missing และ scheduler จะไม่ส่งไปนั้น
- ชื่อคอลัมน์ต้องไม่มีจุด เพราะจุดใช้แบ่ง path ให้ตั้ง alias ใน SQL แทน
- connector จะแปลง date / datetime เป็น ISO string และ Decimal เป็นตัวเลข JSON ให้ตรง contract; ยังต้องยืนยันผลกับ SQL จริงแบบ read-only (ดู F3)

ตัวอย่างโครง config (ชื่อ view และคอลัมน์เป็นตัวอย่าง ต้องแทนด้วยผล Phase 01):

```
"sqlserver": {
  "orders_query": "SELECT h.so_no AS order_no, CONVERT(char(10), h.so_date, 23) AS order_date, h.credit_days AS payment_in_day,
h.cust_code AS customer_code, h.ship_addr AS shipping_address, h.hub_code AS pickup_hub_code, l.line_no AS line_no, l.item_code AS
item_code, l.item_name AS description, CAST(l.qty AS float) AS quantity, CAST(l.price AS float) AS unit_price FROM dbo.v_so_ready h JOIN
dbo.v_so_lines l ON l.so_no = h.so_no WHERE h.so_no = ?",
  "order_key": "order_no",
  "field_map": {"order_no": "order_no", "order_date": "order_date", "payment_in_day": "payment_in_day", "customer.code":
"customer_code", "shipping_address": "shipping_address", "pickup_hub_code": "pickup_hub_code"},
  "item_field_map": {"line_no": "line_no", "item_code": "item_code", "description": "description", "quantity": "quantity", "unit_price":
"unit_price"}
}
```

9.0.1 ฟิลด์ที่ส่งให้ eVRP จริง (อัปเดต 29 ก.ย. 2026)

query ที่ใช้งานอยู่สร้างจาก query ของทีม ERP เอง อ่านสี่ตาราง

ตาราง	ทำอะไร
SalesOrderHdr	เลขที่ วันที่ ลูกค้า ที่อยู่ส่ง Company Comname IncludeVat LocationCode DeliveryDateHdr
SalesOrderDtl	บรรทัดสินค้า จำนวน ราคา LotNo
Customer	ที่อยู่เดิม อำเภอ จังหวัด รหัสไปรษณีย์ เบอร์โทร Google Maps ชื่อเซลล์
InventoryItem	M3 NW MainUnits PackUnits

จุดที่เคยเข้าใจผิดและแก้แล้ว

- **น้ำหนักและปริมาตรอยู่ที่ InventoryItem ไม่ใช่บรรทัดออเดอร์** — เดิมอ่าน SalesOrderDtl.NW/M3

ซึ่งว่างเสมอ ทำให้ nw / cbm ที่ส่งไป eVRP เป็น null ทุกใบ

- ** VatType ของ ERP คือคอลัมน์ IncludeVat ** — ชื่อไม่ตรงกัน
- ** DeliveryDateHdr มีอยู่จริง** จึงใช้ค่านั้นก่อน แล้วค่อยตกไปที่กฎ +1 วันข้ามวันอาทิตย์
- ** GW และ lotno รองรับแล้วตั้งแต่ VRP API v1.3 (30 ก.ย. 2026)** — เราส่งรอไว้ก่อนที่เขาจะเปิด

จึงใช้ได้ทันทีที่ v1.3 ออก แต่ **ชื่อฟิลด์คือ lotno ไม่มีชิตล่าง** ต่างจากที่เราตั้งไว้ตอนแรก (lot_no) ถ้าไม่แก้ eVRP จะทิ้งค่าเจียบๆ โดยไม่แจ้ง error

- ** lotno ยังว่างทุกบรรทัด** — คลังเลือก lot ตอนหยิบของ ไม่ใช่ตอนเปิด SO

(ตรวจแล้ว 0 จาก 3,455 บรรทัด) ค่าจริงจะกลับมาทาง DO แทน

- ** pickup_hub_code ยังเป็นค่าคงที่ TS-h01 ** ไม่ใช่ LocationCode ของ ERP เพราะ eVRP ผูกรหัส hub

กับ master ของเขา การส่งรหัสคลัง ERP ตรงๆ จะได้ PICKUP_HUB_NOT_FOUND ทั้งหมด

company, comname, vat_type ไม่ได้ใช้จัดเส้นทางที่ eVRP — เขาเก็บไว้เฉยๆ (v1.2 §7) แล้วส่งกลับมา กับ DO ซึ่งเป็นวิธีเดียวที่ใบส่งของจะรู้ว่า เป็นของบริษัทไหน

9.0.1.1 กฎน้ำหนักและ lot ของ eVRP v1.3

- `** gw` มีลำดับก่อน `nw` `**` — ถ้า `gw > 0` ระบบจะใช้ `gw` เป็นน้ำหนักต่อหน่วย ถ้าไม่มีหรือเป็น 0
- จึงตกไปใช้ `nw` (v1.3 §5)
- ใช้เฉพาะตอน `**สร้างสินค้าใหม่อัตโนมัติ**` ไม่ทับน้ำหนักของสินค้าที่มีอยู่แล้วในทะเบียนของเขา
 - `** lotno` ผูกกับบรรทัดสินค้า ไม่ใช่ทะเบียนสินค้า `**` สินค้ารหัสเดียวกันส่งหลายบรรทัดได้ถ้าคนละ lot
- และเขาจะไม่รวมบรรทัดให้ เก็บไว้ที่ `neo_so_product.sop_lot_no` (v1.3 §6) — ตรงกับที่เราขอให้เขา แดกบรรทัดตอนส่ง DO กลับ
- ยาวได้ถึง 100 ตัวอักษรฝั่งเขา แต่ `tbl_Dodtl.LotNo` ของ ERP กว้าง 50

9.0.2 ขอบเขตบริษัท

ส่งครบทุกบริษัทแล้วตั้งแต่ 29 ก.ย. 2026 — ไม่มีเงื่อนไขกรองบริษัทใน query อีก

ผลการส่งจริงครั้งแรกด้วย Token ตัวจริง: `**ส่งสำเร็จ 900 ใบ ถูกปฏิเสธ 478 ใบ (65%)**` ทุกใบที่ถูก ปฏิเสธเกิดจาก Master ฝั่ง eVRP ไม่ใช่รูปแบบข้อมูลของเรา — ต้องสร้างจุดส่ง 312 จุด, ลูกค้า 45 ราย และผูก Route อีก 1 จุด รายชื่อส่งให้ทีม eVRP แล้ว

`sync.max_new_orders_per_cycle` ตั้งไว้ 100 ใบต่อรอบ ปกติไม่มีผลเพราะวันหนึ่งมี SO ใหม่ไม่กี่สิบใบ แต่จะจำกัดความเร็วเมื่อมีงานค้างจำนวนมาก เช่นตอนขยายขอบเขต query

บริษัท	SO อนุมัติ (29 ก.ย. 2026)
NSF	1,259
BNS	64
TST	47
NST	8

ตรวจการเชื่อมต่อแบบอ่าน metadata อย่างเดียว (ไม่แตะตารางธุรกิจ):

```
docker compose -f deploy/docker-compose.yml exec thaisausage python -c "import json; from thaisausage.sqlserver import SQLServerConnector; c=json.load(open('/app/config/local.json')); print(SQLServerConnector(c['sqlserver']).check_read_only_connection())"
```

9.1 DO writer เข้า ERP (‘do_write’)

สถานะ: VERIFIED — mapping ครบ 79 คอลัมน์ และเขียนเข้า ERP จริงแล้ว 2 ใบ ตรวจกลับตรงทุกคอลัมน์ (ดู `process/features/erp-sqlserver/reports/do-write-18-09-26.md`) mapping ที่ใช้อยู่ใน `config/do-write-template.json` ส่วนที่มาของแต่ละคอลัมน์อธิบายไว้ใน `docs/eVRP_D0_Callback_Spec.md §9`

`** TransactionNo` ไม่ใช่ identity และไม่มี default `**` — ต้องมีคนออกเลข แอป ERP ใช้ MAX+1 และ ระบบเราก็เคยใช้แบบเดียวกันจนเกิดเลขชนกันจริงเมื่อ 18 ก.ย. 2026 ปัจจุบันจึงนับเลขเฉพาะในช่วงที่ทีม ERP กันไว้ให้ (`transaction_no_minimum: 900000`) และตรวจก่อน commit ว่าเลขนั้นมีเอกสารเดียว

กติกาลดความปลอดภัยที่บังคับในโค้ด:

- ค่าเริ่มต้น `do_write.enabled=false` และ `do_write.update_enabled=false` เปิดได้จาก config เท่านั้น ห้ามเปิดจาก payload
- การแก้ไขเอกสารเดิมมี flag แยกของตัวเอง เพราะการแทนที่รายการสินค้าคือคำสั่ง `DELETE` จริงกับ ERP
- ต้องใช้ credential คนละชุดกับบัญชี read-only ถ้าตั้ง env เดียวกับ `ERP_SQL_USER / ERP_SQL_PASSWORD / ERP_SQLSERVER_CONNECTION_STRING` ระบบจะปฏิเสธ
- validate mapping และ payload ให้เสร็จก่อนเปิด connection เสมอ
- ชื่อตารางและคอลัมน์มาจาก config ที่ review แล้ว ค่าทุกค่าถูกผูกเป็น parameter ไม่มีการต่อ string จาก payload
- Header และ Detail อยู่ใน transaction เดียว ถ้าบรรทัดใดล้มเหลวจะ rollback ทั้งชุด
- `EntryDate` ใช้ `GETDATE()` ของ SQL Server ส่วน `IsAcc=0` , `IsAccBy` , `IsAccDate` , `DocuNw` เป็นค่าคงที่ตาม contract
- เมื่อ `dry_run=true` ระบบจะ validate และนับ statement เท่านั้น ไม่เปิด connection

สถานะที่บันทึกในตาราง `do_writes`:

State	ความหมาย	เขียนซ้ำได้
preview	dry-run ตรวจ mapping แล้ว ไม่ได้เขียน	ได้
disabled	flag ปิดอยู่	ได้
rejected	validate ไม่ผ่าน ไม่มี SQL ถูกส่ง	ได้ (หลังแก้สาเหตุ)
rehearsed	รัน INSERT จริงแล้ว rollback — พิสูจน์สิทธิ์และ mapping โดยไม่ทิ้งข้อมูล	ได้
failed	SQL Server ปฏิเสธก่อน commit และ rollback แล้ว ไม่มีอะไรค้างใน ERP	ได้ (หลังแก้สาเหตุ)
inserted	commit สำเร็จ	ไม่ได้
needs_review	ผลไม่แน่นอน เช่น timeout ระหว่าง commit	ไม่ได้ ต้องตรวจ ERP ด้วยมือก่อน

เส้นแบ่งอยู่ที่ commit: อะไรที่ล้มก่อน commit ฐานข้อมูล rollback ให้แล้ว จึงลองใหม่ได้ปลอดภัย ส่วน `needs_review` คือไม่รู้ว่าจะข้อมูลเข้าไปหรือยัง การลองใหม่อาจได้เอกสารสองใบ

ตาราง `do_updates` เก็บสถานะการแก้ไขเอกสารแยกต่างหาก ใช้สถานะชุดเดียวกันโดยมี `updated` แทน `inserted`

การกันซ้ำใช้ `receipt_id` เป็น primary key พร้อม unique index บน `do_no` และ `transaction_no` ถ้า payload ของ receipt เดิมเปลี่ยนหลังเคยเขียนแล้วจะได้ conflict ไม่ใช่การเขียนซ้ำ

10. ข้อมูลใน SQLite

```
submissions(request_id PK, payload_hash, state, result JSON, created_at)
order_claims(order_no PK, request_id)
erp_hooks(event_id PK, event_type, source_id, company_id, order_no, changed_at, state, received_at)
do_receipts(receipt_key PK, source, receipt_id, do_no, payload_hash, payload, state, received_at)
do_writes(receipt_key PK, source, receipt_id, do_no UNIQUE, transaction_no UNIQUE, payload_hash, state, reason, updated_at)
```

ใน container ไฟล์ฐานข้อมูลอยู่ที่ `/app/data/integration.sqlite3` บน volume `thaisausage-data` ตาราง `submissions` เก็บเฉพาะ hash, สถานะ และเลขอ้างอิงจาก eVRP ไม่ได้เก็บ payload เต็ม ดังนั้นจึงต้องเก็บไว้ที่ ERP เพื่อใช้ reconcile

ดูจำนวนรายการตามสถานะ:

```
docker compose -f deploy/docker-compose.yml exec thaisausage python -c "import sqlite3; db=sqlite3.connect('data/integration.sqlite3');
print('hooks', db.execute('SELECT state, count(*) FROM erp_hooks GROUP BY state').fetchall()); print('submissions', db.execute('SELECT
state, count(*) FROM submissions GROUP BY state').fetchall())"
```

สำรองข้อมูลโดยใช้ SQLite backup API ซึ่งทำได้ขณะระบบทำงาน:

```
docker compose -f deploy/docker-compose.yml exec thaisausage python -c "import sqlite3;
sqlite3.connect('data/integration.sqlite3').backup(sqlite3.connect('/tmp/backup.sqlite3'))"
docker compose -f deploy/docker-compose.yml cp thaisausage:/tmp/backup.sqlite3 ./integration-$(date +%F).sqlite3
```

ห้ามลบแถวใน `order_claims` เพื่อบังคับส่งซ้ำ และห้าม restore backup เก้าสิบขณะเปิดโหมด live ก่อน reconcile กับ eVRP

11. Deploy บน VPS

Service	รายละเอียด
thaisausage	Python 3.11 slim + ODBC Driver 18 รันด้วย user <code>appuser</code> (uid 10001) เปิดพอร์ต 8080 เฉพาะใน network ภายใน root filesystem เป็น read-only, <code>/tmp</code> เป็น tmpfs และมี healthcheck เรียก <code>/health</code> ทุก 30 วินาที
caddy	Caddy 2 รับพอร์ต 80/443 ออก HTTPS อัตโนมัติสำหรับ <code>thaisausage.krs.co.th</code> แล้ว reverse proxy ไปที่ app และเริ่มหลังจาก app healthy แล้ว
Volumes	<code>thaisausage-data</code> (SQLite), <code>caddy-data</code> , <code>caddy-config</code>
Mount	<code>config/local.json</code> แบบ read-only และ <code>.env</code> ผ่าน <code>env_file</code> (format raw)

ข้อกำหนดก่อน deploy:

- DNS A record ของ `thaisausage.krs.co.th` ต้องชี้มาที่ VPS และพอร์ต 80/443 ต้องเปิดเพื่อให้ออก certificate ได้
- image ถูก build สำหรับ amd64 เพราะ repo ของ Microsoft ระบุ `arch=amd64` ไว้ตายตัว
- build context คือ root ของ repo จึงต้องใช้ `.dockerignore` ที่ root เท่านั้น (ไฟล์ใน `deploy/` ไม่มีผล) ไฟล์นั้น `.env`, `config/local.json`, `deploy/vps.env`, `data/`, `tests/`, `process/` และ PDF ไม่ให้เข้า build context
- ต้องมี `/opt/thaisausage/.env` (permission 600) และ `config/local.json` บน VPS

Deploy ครั้งแรกหรืออัปเดต:

```
cd /opt/thaisausage
git fetch origin main
git reset --hard origin/main # ทิ้งการแก้ไขไฟล์ที่ track บน VPS; .env และ config/local.json ถูก ignore จึงไม่หาย
docker compose -f deploy/docker-compose.yml build
docker compose -f deploy/docker-compose.yml up -d
docker compose -f deploy/docker-compose.yml ps
curl -fsS https://thaisausage.krs.co.th/health
```

ก่อนอัปเดตให้สำรอง SQLite (หัวข้อ 10) เสมอ หลังอัปเดตให้ตรวจ `/health`, `/docs`, log และลองส่ง webhook กับ DO แบบ mock ได้แต่ที่ยังไม่ commit และ push จะไม่ขึ้นไปบน VPS ดังนั้น ณ วันที่เขียนเอกสาร ระบบบน production ยังไม่มีการปลุก sweep ด้วย webhook, การรองรับ HEAD, favicon และการแก้ CSP ของ Swagger

12. การดูแลประจำวันและการแก้ปัญหา

12.1 รายการตรวจประจำวัน

- [] `docker compose ps` ต้องแสดงทั้งสอง service เป็น running และ app เป็น healthy
- [] `https://thaisausage.krs.co.th/health` ต้องตอบ 200 และค่า `dry_run` ต้องตรงกับที่อนุมัติไว้
- [] นับ hook ที่เป็น `received` ค้างนาน และ hook ที่เป็น `review`
- [] นับ submission ที่เป็น `sending` หรือ `needs_review`
- [] ตรวจพื้นที่ดิสก์ของ volume และตรวจว่ามี backup ล่าสุด
- [] ตรวจอายุ certificate ของ Caddy

12.2 อาการและวิธีแก้

อาการ	สาเหตุที่เป็นไปได้	วิธีแก้
ระบบไม่เริ่ม: <code>dry_run must explicitly be true or false</code>	ค่าใน config ไม่ใช่ boolean	แก้เป็น <code>true</code> หรือ <code>false</code> โดยไม่มีเครื่องหมายคำพูด
ระบบไม่เริ่ม: Set the environment variable named by <code>api_key_env</code>	ไม่มี <code>THAISAUUSAGE_API_KEY</code>	ตรวจ <code>.env</code> หรือ <code>env_file</code> แล้ว restart
ระบบไม่เริ่ม: Live mode requires the VRP token	<code>dry_run=false</code> แต่ไม่มี <code>VRP_TOKEN</code>	ใส่ token หรือกลับไปใช้ <code>dry-run</code>
ระบบไม่เริ่ม: <code>KeyError</code>	config ขาด key หลัก เช่น <code>vrp</code> หรือ <code>erp</code>	เทียบกับ <code>config/example.json</code>
401 unauthorized	key ผิด หรือขาดคำว่า <code>Bearer</code>	ตรวจ header
422 Content-Type must be application/json	client ส่ง form หรือ chunked	ตั้ง header และส่ง Content-Length
502 <code>erp_request_failed</code>	ERP REST ไม่ตอบ, ไม่ใช่ HTTPS หรือเจอ redirect	ตรวจ <code>erp.base_url</code> และ token (ระบบไม่ follow redirect)
hook ค้างที่ <code>received</code>	<code>sync.enabled=false</code> หรือ sweep error ทั้งรอบ	เปิด sync ตาม gate และตรวจ SQL config และ driver
hook เป็น <code>review</code>	ไม่พบ SO, SQL error, validate ไม่ผ่าน, <code>order_no</code> ซ้ำ, <code>order_no</code> หาย หรือข้อมูลเก่าที่ไม่ทราบ hash	ทดสอบ query ด้วย <code>order_no</code> นั้นแบบ read-only แล้วตรวจ reason/mapping
submission เป็น <code>needs_review</code>	eVRP timeout, error หรือตอบไม่ตรงรูปแบบ	อ่าน <code>upstream_error</code> ในผลลัพธ์ก่อน แล้วตรวจที่ eVRP ด้วยเลข SO; ห้ามส่ง payload ที่เปลี่ยนแล้วซ้ำด้วย <code>request_id</code> เดิม
<code>upstream_http_422 + PICKUP_HUB_NOT_FOUND</code>	รหัส hub ที่ส่งไปไม่มีใน Master Hub ของ eVRP	ขอรหัสจริงจาก eVRP แล้วแปลงใน <code>approved_orders_query</code> ด้วย CASE
<code>upstream_http_422 + CUSTOMER_MASTER_REQUIRED</code>	ลูกค้ายังไม่มี master/จุดส่ง/Route ในระบบ eVRP	ให้ eVRP สร้าง master ก่อน ระบบเราแก้เองไม่ได้
<code>upstream_http_409 + REQUEST_ID_CONFLICT</code>	ส่ง <code>request_id</code> เดิมด้วยข้อมูลที่แก้แล้ว	ปกติระบบสร้าง id ใหม่ให้อัตโนมัติเมื่อข้อมูลเปลี่ยน; ถ้ายังชนแปลว่า payload เหมือนเดิมจริง
<code>/docs</code> หน้าว่าง	browser โหลด <code>unpkg.com</code> ไม่ได้	ใช้ <code>/openapi.json</code> กับ Postman หรือ Swagger Editor แทน
app unhealthy	process ค้าง หรือ config ผิด	<code>docker compose logs --tail=100 thaisausage</code>
HTTPS ใช้ไม่ได้	DNS ผิด หรือพอร์ต 80/443 ถูกปิด	ตรวจ DNS, firewall และ <code>docker compose logs caddy</code>

12.3 ค่าจริงที่ eVRP ยืนยันแล้ว (18 ก.ย. 2026)

เรื่อง	ค่าที่ eVRP ยืนยัน
pickup_hub_code	** TS-h01 ** ใช้ได้ทั้งคลังนครปฐมและกรุงเทพ จึงตั้งเป็นค่าคงที่ในquery ไม่ต้องแปลงจาก LocationCode
delivery_point_code	**เท่ากับรหัสลูกค้า** — eVRP สร้างจุดส่งโดยใช้รหัสลูกค้าของเราเป็นรหัสจุดส่ง
delivery_date	ให้เราส่งมาเสมอ คำนวณเป็น order_date + 1 วัน และถ้าตรงวันอาทิตย์ให้ข้ามไปวันจันทร์* (query ใช้ DATEDIFF จาก 1900-01-07 จึงไม่ขึ้นกับ DATEFIRST หรือภาษาของ server)
Customer master	ของ TST ยังขาด CT-LR-BKK-2088 และ CT-LR-BKK-2093 · ทั้งระบบต้องการ 940 ราย ส่งรายชื่อให้ eVRP แล้ว 29 ก.ย.

12.3.2 อัปเดตจากคู่มือ VRP API v1.2 (26 ก.ย. 2026)

- **X-Token** กำหนด **com_id** และการค้นหา Hub / Customer / POI ทั้งหมดถูกจำกัดอยู่ใน company นั้น
- ถ้าจะส่งของหลายบริษัท ต้องรู้ก่อนว่า eVRP ใช้ com_id เดียวหรือแยก ถ้าแยกต้องขอ token เพิ่ม และระบบเราต้องแก้ไขให้เลือก token ตามบริษัทของแต่ละใบ (ตอนนี้รองรับ token เดียว)
- ห้ามใช้ **Authorization: Bearer** กับ endpoint นี้ (เราใช้ **X-Token** อยู่แล้ว)
 - **pickup_hub_code** กำหนด hub เจ้าของ SO** ไม่ใช่แค่จุดรับของ และ hub นั้นต้องมี Default Pickup POI
 - **payment_in_day** **0 = COD (cod_type=1) · ค่าอื่นหรือ null = ไม่ COD**
 - **company / comname / vat_type** ** เก็บอย่างเดียว ไม่กระทบ routing, VAT, COD — สงวนไว้สำหรับ

DO callback ในอนาคต ซึ่งเป็นเหตุผลที่เราส่งไปตั้งแต่ 29 ก.ย.

- **lotno** และ **gw** เพิ่มเข้ามาใน v1.3** (30 ก.ย. 2026) ดูหัวข้อ 9.0.1.1

12.3.1 ข้อกำหนดฝั่ง eVRP (ยืนยันกับคู่มือ VRP Integration Guide v1.1 แล้ว)

- กฎด้านล่างอยู่ในคู่มือของผู้ให้บริการหัวข้อ 2.2 (Idempotency), 2.10 (HTTP Status Codes) และ 2.11 (Validation Rules) และเราเจอข้อตอนส่ง SO ในจริงในแรก:
- **eVRP** จอง **request_id** ไว้แม้จะปฏิเสธ payload นั้น** ส่ง id เดิมพร้อมข้อมูลที่แก้แล้วจะได้ **409 REQUEST_ID_CONFLICT** ระบบจึงสร้าง **request_id** จาก **order_no** บวก hash ของเนื้อข้อมูล เพื่อให้ข้อมูลที่แก้แล้วได้ identity ใหม่เสมอ
 - **รหัส hub** ต้องมีใน Master Hub ของ eVRP** รหัสคลังของ ERP (L01 กทม., L03 นครปฐม) ไม่ใช่รหัสเดียวกับของ eVRP ต้องแปลงใน query
 - **ลูกค้าต้องมี master + จุดส่ง + Route ในระบบ eVRP ก่อน** มิฉะนั้นได้ CUSTOMER_MASTER_REQUIRED** เป็นงานฝั่ง eVRP ล้วน ระบบเราตรวจสอบล่วงหน้าไม่ได้
 - **delivery_date** วางได้** eVRP จะ default เป็น **order_date** แล้วตอบ warning **DELIVERY_DATE_DEFAULTED**
 - **จุดส่งต้องมี Route ส่วนตัวลูกค้าไม่ต้องมี** และ delivery_point_code** ถ้าส่งไปต้องตรงกับจุดส่งที่ VRP รู้จัก ถ้าไม่ส่งระบบจะใช้ **shipping_address** ระบุจุดส่งแทน
 - ซิดจำกัดอื่นตามคู่มือ: body ไม่เกิน 10 MB (413), ต้องเป็น application/json (415), **order_no** ห้ามซ้ำภายใน request เดียวกัน
 - **ระวางการแปลงค่าว่าเป็นตัวเลข**:** ใน ERP **Paymentinday** เป็น nvarchar(10) และ TRY_CONVERT(float, '') คืนค่า **0** ซึ่ง eVRP อ่านว่า COD — query จึงต้องใช้ **NULLIF(LTRIM(RTRIM(...)), '')** ก่อนแปลงเสมอ มิฉะนั้นคนขับจะไปเก็บเงินปลายทางผิดไป
 - **description** ของทุกบรรทัดเป็น Required ของ eVRP** query จึง fallback เป็น **ItemName → Description → ItemCode** และ validator ของเราบังคับด้วยอีกชั้น
 - ข้อความปฏิเสธจาก eVRP ถูกเก็บไว้ในฟิลด์ **upstream_error** ของ submission (สูงสุด 2000 ตัวอักษร) อ่านได้โดยไม่ต้องส่งซ้ำ

12.4 การล้าง claim เพื่อส่งใหม่

order_claims กันการส่งซ้ำ ระบบจะไม่ส่งใบเดิมอีกเองไม่ว่ากรณีใด การล้าง claim ทำได้เฉพาะเมื่อ**ยืนยันแล้วว่า eVRP ไม่ได้รับ**:

- ปลอดภัย: **upstream_http_422** หรือ **409** — eVRP ปฏิเสธชัดเจน ไม่มีข้อมูลค้างฝั่งเขา
- **ห้าม**:** **upstream_outcome_unknown** หรือ timeout — ต้องตรวจกับ eVRP ด้วยเลข SO ก่อนเสมอ

```
docker compose -f deploy/docker-compose.yml exec -T thaissausage python -c "
import sqlite3
db = sqlite3.connect('data/integration.sqlite3')
with db:
    db.execute("\DELETE FROM order_claims WHERE TRIM(order_no)='SO-XXXX'\")
print('cleared')"
```

12.5 ขั้นตอนเปิดใช้งานจริง (ย่อ)

- Gate A: ติดตั้ง ODBC แล้ว, ผ่าน metadata check (หัวข้อ 9) และบัญชี SQL เป็น read-only จริง

- Gate B: ใส่ approved_orders_query และ mapping แล้ว ทดสอบกับ SO ที่อนุมัติใน dry-run ต้องได้ JSON ถูกต้องครบทุกกรณี (COD, credit, ของแถม, หลายบรรทัด, ภาษาไทย, ทศนิยม)
- Gate C: ทดสอบกับ eVRP UAT โดยใช้ token ทดสอบ แล้วตรวจเลขอ้างอิงและยอดเงิน
- Gate D: ซ้อม backup/restore, กำหนดผู้รับผิดชอบ แล้วตั้ง dry_run=false ก่อน จากนั้นจึงตั้ง sync.enabled=true และเฝ้าดูรอบแรก
- F1-F3 แก่แล้วและมี regression test; Gate B ยังต้องผ่านการเชื่อม SQL จริงแบบ read-only กับ SO ที่อนุมัติ รายละเอียดอยู่ใน Operations Guide และ Integration Test Plan

12.6 เครื่องมือตรวจก่อนส่ง (`preflight`)

รันได้ทุกเมื่อ อ่านอย่างเดียว ไม่ติดต่อ eVRP และไม่เขียนอะไรทั้งสิ้น:

```
docker compose -f deploy/docker-compose.yml exec -T thaisausage \
python -m thaisausage.preflight --config /app/config/local.json --all-orders
```

รายงาน 5 ส่วน:

- ****config**** — dry_run, sync.enabled, do_write.enabled, มีเงื่อนไข IsApprSo = 1 โหม, ค่า TOP, ตัวกรองบริษัท, hub mapping และตัวกรองใบทดสอบที่อาจลืมนोट
- ****data**** — จำนวน SO/บรรทัด, แยก COD / เครดิต / ไม่ใช่ COD, order_no ซ้ำหรือมีช่องว่างติดมา
- ****erp reconciliation**** — เทียบ $\Sigma(\text{quantity} \times \text{unit_price})$ กับยอดในหัว SO (ต้องตั้ง sqlserver.totals_query ก่อน) ถ้าไม่ตรงแปลว่า map คอลัมน์ผิดหรือบรรทัดหาย
- ****needs to exist in the eVRP master**** — รายการ hub / customer / delivery point ที่ต้องมีในระบบ eVRP
- ****contract**** — validate ทุกใบเหมือนตอนส่งจริง พร้อมจับ hub ที่ยังเป็น placeholder (HUBCODE, XXX, REPLACE, ...)

ปิดท้ายด้วย READY / NOT READY และ exit code 0/1 จึงใส่ใน cron หรือขั้นตอน pre-deploy ได้ ผลลัพธ์มีแต่รหัสเอกสารและตัวเลข ไม่มีข้อมูลลูกค้า

12.7 เครื่องมือสำรวจ schema (`discover`)

ใช้ตอนต้องรู้ชนิดข้อมูล/คีย์/trigger ของตาราง ERP หรือหาว่าคอลัมน์หนึ่งอยู่ตารางไหน:

```
docker compose -f deploy/docker-compose.yml exec -T thaisausage \
python -m thaisausage.discover --config /app/config/local.json \
--tables dbo.tbl_D0hdr,dbo.tbl_Dodtl --find-column IsApprSo --out /app/data/discovery.json
```

อ่านเฉพาะ INFORMATION_SCHEMA และ sys.* ผ่าน SELECT guard เดิม ไม่แตะข้อมูลธุรกิจ เขียนผลลง /app/data/ (volume) เพราะ /tmp เป็น tmpfs ที่ docker cp อ่านไม่ได้

12.8 หา query ที่รันไม่ได้

ถ้า log ขึ้น scheduled sweep cycle finished: orders=1 [('review', 1)] ทุกวันที่ แปลว่าอ่านข้อมูลไม่ได้ เลย ไม่ใช่กวาดแล้วไม่มีของ ดูสาเหตุที่บรรทัดถัดไป

```
docker compose -f deploy/docker-compose.yml logs --since 5m thaisausage | grep -i "could not read"
```

42S22 = ชื่อคอลัมน์ผิด · 42S02 = ชื่อตารางผิด · ทั้งสองแบบจะบอกข้อที่ผิดมาด้วย

****เกิดขึ้นจริงเมื่อ 29 ก.ย. 2026:**** เพิ่ม h.VatType เข้า query โดยไม่ได้ตรวจว่าคอลัมน์นั้นมีอยู่จริง (ชื่อจริงคือ IncludeVat) SO หยดไหลทั้งหมดกว่าชั่วโมง วิธีแก้คือรัน preflight หลัง assemble_config.py ทุกครั้งก่อนปล่อยให้กวาด และถ้าล้ม **ย้อนไฟล์สร้างก่อน อย่า restart ทั้**

12.9 สรุปสถานะประจำวัน (`status`)

ตอบคำถามที่ต้องถามทุกเช้า: ตัวกวาดยังทำงานไหม ส่ง SO ไปกี่ใบ อะไรค้าง DO เข้ามาก็ใบ อะไรต้องให้คนดู

```
docker compose -f deploy/docker-compose.yml exec -T thaisausage \
python -m thaisausage.status --config /app/config/local.json --days 7
```

อ่านจาก SQLite ของเราอย่างเดียว ไม่ต่อ ERP ไม่ต่อ eVRP รันได้ตลอดเวลา สถานะที่ต้องให้คนดูจะมี << กำกับ และ SO ที่ส่งไม่สำเร็จจะแสดงเลข SO พร้อมรหัสและข้อความ error จาก eVRP ตรงๆ ไม่ใช่แค่ว่าล้มเหลว

12.10 ตรวจเอกสารที่เขียนไปแล้ว (`verify\_do`)

จำนวนแถวบอกได้ความมีข้อมูลเข้าไป ไม่ได้บอกว่าค่าลงถูกช่อง คำสั่งนี้อ่านเอกสารกลับจาก ERP แล้ววางเทียบ กับสิ่งที่เราส่งทีละคอลัมน์

```
docker compose -f deploy/docker-compose.yml exec -T thaisausage \
python -m thaisausage.verify_do --config /app/config/local.json \
--transaction-no 900000 --receipt-id TEST-ERP-002
```

ใช้ credential ฝั่งอ่านและเป็น SELECT ธรรมดาผ่าน guard เดิม TransactionNo ผูกเป็น parameter การเทียบดูที่ความหมายไม่ใช่ตัวอักษรดิบ — ERP เดิมช่องว่างท้ายข้อความ คินเงินเป็น Decimal และวันที่เป็น datetime จึงไม่พ้องผิดพลาดลง คอลัมน์ที่ว่างทั้งสองฝั่งถูกซ่อน เดิม --all-columns เพื่อดูครบทั้ง 79 ตัดคำสั้นย่อย --receipt-id ออกได้ จะกลายเป็นแค่เปิดดูเอกสารใน ERP ใช้ตรวจใบที่ทีม ERP คีย์เองก็ได้

12.11 ประกอบ config กลับจาก git (`assemble\_config`)

config/local.json ไม่ได้อยู่ใน git เพราะแก้ที่เครื่องจริง แต่สองซีกของมันอยู่ใน git ทั้งคู่ — config/production.json (ค่าตั้งค่าและ SQL ที่ผ่านการตรวจ) กับ config/do-write-template.json (mapping)

```
python3 deploy/assemble_config.py          # สร้าง config/local.json ใหม่
python3 deploy/assemble_config.py --check  # เทียบว่าที่ใช้อยู่ตรงกับ git ใหม่
```

ไฟล์ที่ประกอบได้จะปิดการเขียน DO ไว้เสมอ เครื่องที่เพิ่งกู้คืนจึงไม่เริ่มเขียน ERP เอง ทั้งสองไฟล์เก็บแค่ *ชื่อ* ตัวแปรสภาพแวดล้อม ไม่มีรหัสผ่านอยู่ใน git

13. การทดสอบ

```
python3 -m unittest discover -s tests -v
```

ผลวันที่ 17 ก.ย. 2026: ผ่าน 123 จาก 123 test ครอบคลุม HTTP auth, validation, idempotency, การส่งซ้ำพร้อมกัน, timeout, DO staging/callback (replay, conflict, concurrent), scheduled sweep, SQL guard, credential ของ SQL และชุดทดสอบ DO writer (parameter binding, transaction, rollback, duplicate, dry-run, flag ปิด และการสร้าง statement ตามคอลัมน์จริงของ ERP) ทุก test ใช้ mock โดยไม่เรียก ERP หรือ eVRP จริง และไม่มี INSERT เข้า ERP เกิดขึ้น

ส่วนที่ยังไม่มี test เชื่อม SQL Server จริงคือ connection/query กับ schema ของ ERP; มี unit test สำหรับ _group_rows และชนิดข้อมูล Decimal/date แล้ว แต่ยังต้องทำ read-only verification กับ SO ที่อนุมัติ

14. ผลการตรวจสอบโปรเจค

14.1 ประเด็นที่ตรวจพบและสถานะแก้ไข

ID	ระดับ	ประเด็น	ตำแหน่ง	ผลกระทบ / หลักฐาน	ข้อเสนอ
F1	สูง	mapped ไม่ได้ import	thaisausage/sqlserver.py	แก้แล้ว: import โดยตรงและมี regression test <code>_group_rows</code>	
F2	สูง	sweep ใน dry-run ทำให้ hook ถูกใช้ไปแล้ว	service.py	แก้แล้ว: dry-run คง Hook เป็น <code>received</code> พร้อม reason <code>dry_run_preview</code>	
F3	สูง	ชนิดข้อมูลจาก pyodbc (Decimal, date) ไม่ผ่าน validate	sqlserver.py	แก้แล้ว: normalize เป็น JSON-compatible type และมี unit test; ยังต้องทดสอบ SQL จริงแบบ read-only	
F4	กลาง	hook ที่ไม่มี <code>order_no</code> ทำให้ query ไม่มีเงื่อนไข	service.py	แก้แล้ว: ตั้ง <code>review/order_no_required</code> และไม่เรียก SQL	
F5	กลาง	ชุดอักขระของ <code>event_id</code> ไม่ตรงกับ <code>request_id</code>	service.py	แก้แล้ว: validate ด้วย pattern เดียวกับ <code>request_id</code>	
F6	กลาง	sweep error ถูกกลืนเจียบๆ และไม่เก็บ reason	<code>__main__.py</code> , <code>service.py</code>	แก้แล้ว: log แบบไม่เปิดเผย payload และเก็บ reason ต่อ Hook	
F7	กลาง	hook ซ้ำไม่ได้เทียบ body	service.py	แก้แล้ว: เก็บ payload hash และตอบ 409 เมื่อ <code>event_id</code> เดิมมีข้อมูลต่างกัน	
F8	ต่ำ	config หลาย key ไม่มีผลกับการทำงาน	config/example.json	ทำให้เข้าใจผิดว่ามีการควบคุม เช่น <code>encrypt</code> หรือ <code>approved_objects</code>	implement การตรวจหรือทำเครื่องหมายว่ายังไม่ใช้
F9	ต่ำ	โค้ดและเอกสารยังไม่ commit	<code>api.py</code> , <code>__main__.py</code> , <code>docs/</code>	production ต่างจาก repo และเอกสารยังไม่อยู่ใน Git	review แล้ว commit และ deploy
F10	ต่ำ	Swagger โหลจาก unpkg	api.py	แก้แล้ว: pin เป็น 5.32.15 ; ยังควรพิจารณา vendor asset/SRI ใน hardening รอบถัดไป	
F11	ต่ำ	SELECT guard เป็นการตรวจข้อความ	sqlserver.py (<code>select_approved</code>)	แก้แล้ว: บังคับคำสั่งเดียว ห้าม <code>;</code> /comment และเพิ่ม CREATE/GRANT/REVOKE/DENY/BACKUP/RESTORE/WAITFOR/BULK/OPENROWSET; ยังต้องพึงสิทธิ์ read-only ของ DB	
R1	สูง	guard เดิมยอมให้มีหลาย statement ต่อท้าย SELECT	sqlserver.py	แก้แล้ว: ปฏิเสธ <code>;</code> และ comment พร้อม regression test	
R2	สูง	guard <code>IsApprSo</code> ตรวจแค่ว่ามีค่านี้	sqlserver.py	แก้แล้ว: ต้องมี <code>IsApprSo = 1</code> และห้ามเทียบกับค่าอื่น	
R3	สูง	detail ที่ไม่มี <code>item_code</code> ถูกทิ้งเจียบ	sqlserver.py	แก้แล้ว: ทั้ง SO เป็น <code>review/detail_item_code_missing</code>	
R4	สูง	SO ที่ค้าง <code>needs_review</code> ถูกข้ามเจียบทุกรอบ	service.py	แก้แล้ว: รายงาน <code>needs_review/prior_attempt_*</code> และไม่ส่งซ้ำอัตโนมัติ	
R5	สูง	log อาจพ่นข้อความ driver และ <code>order_no</code> ดิบ	<code>__main__.py</code> , <code>service.py</code>	แก้แล้ว: log เฉพาะ class name, นับ state และ sanitize identifier	
R6	กลาง	isolation ไม่ครอบชั้นอ่าน/map	service.py	แก้แล้ว: batch ที่อ่านไม่ได้คืนผล <code>review</code> แทนการโยน exception	
R7	กลาง	DO callback พร้อมกันได้ HTTP 500	service.py	แก้แล้ว: จอง identity ใน BEGIN IMMEDIATE และมี concurrent test	
R8	กลาง	ERP/eVRP ใช้ <code>receipt_id</code> ซนกัน	service.py, <code>api.py</code>	แก้แล้ว: identity เป็น <code><source>:<receipt_id></code> พร้อม migration	
R9	กลาง	<code>.dockerignore</code> อยู่ผิดตำแหน่ง	<code>.dockerignore</code>	แก้แล้ว: ย้ายไป root และพิสูจน์ด้วย docker build context check	

14.2 จุดแข็งที่พบ

- ใช้ idempotency แบบ transaction (BEGIN IMMEDIATE) มี test การส่งซ้ำพร้อมกัน และไม่ resend อัตโนมัติเมื่อผลไม่แน่นอน
- เทียบ API key แบบ constant-time, จำกัดขนาด body, ปฏิเสธ NaN และปิด access log เพื่อป้องกันข้อมูลรั่ว
- connector บังคับ HTTPS, ไม่ follow redirect และจำกัดขนาด response ที่ 4 MiB
- container รันแบบ non-root บน root filesystem ที่เป็น read-only, ใช้ no-new-privileges และเปิด app เฉพาะใน network ภายใน
- credential ของ SQL แยกจาก connection string และ escape ค่าให้ถูกต้อง

14.3 เอกสารเดิมที่ไม่ตรงกับโค้ด

เอกสาร	ข้อความเดิม	ความจริงตามโค้ด
README.md บรรทัดต้น	SQL connector ยังไม่ได้ implement	มี connector แล้ว แต่รอ query/mapping และ read-only verification
README.md หัวข้อ webhook	SQL sweep worker จะ implement ใน Phase 04	ใช้ scheduled approved-SO worker; Hook ไม่ใช่ช่องทางหลัก
README.md สัญญา API	ทุก endpoint ยกเว้น /health ต้องใช้ key	/docs , /openapi.json และ /favicon.ico ก็เป็นสาธารณะ
Integration_Test_Plan.md ERP-04	event_id เดิมที่ข้อมูลเปลี่ยนจะได้ 409	โค้ดตรวจ payload hash และตอบ conflict
Integration_Test_Plan.md VRP-03	มี controlled retry และ attempt record	ทดสอบว่าตั้ง needs_review และไม่ resend อัตโนมัติ; ยังไม่มี attempt history
Thaisausage_Operations_Guide.md หัวข้อ 3	config/local.json ต้องกำหนด source และ approved query set	โค้ดยังไม่ได้อ่าน key เหล่านี้ (F8)

15. เอกสารที่เกี่ยวข้อง

- docs/ERP_Integration_Guide.md — สำหรับทีม ERP: webhook, JSON contract และ retry
- docs/Thaisausage_Operations_Guide.md — release gates และกติกากาการกู้คืน
- docs/Integration_Test_Plan.md — test case และแบบฟอร์ม sign-off
- docs/erp-source-contract.md — รายการข้อมูลที่ต้องได้จาก ERP/DBA ใน Phase 01
- docs/do-insert-contract.md — mapping matrix ของ tbl_D0hdr / tbl_Dodtl สำหรับ DBA
- process/features/erp-sqlserver/active/PHASE_07_D0_INSERT_PLAN_16-09-26.md — แผน DO Insert
- docs/openapi.json และ https://thaisausage.krs.co.th/docs
- process/features/erp-sqlserver/active/ERP_SQLSERVER_PLAN_12-09-26.md — แผน 6 phase